

Aula 2: Microsserviços, MicroProfile, GraalVM e o Ecossistema Quarkus

Bem-vindos à nossa segunda aula! Após entendermos a base de aplicações web e o protocolo HTTP, hoje vamos mergulhar no mundo moderno do desenvolvimento Java focado na nuvem. Reservem os próximos minutos para absorver estes conceitos antes de colocarmos a mão na massa.

Parte 1: Arquitetura e Tecnologias de Base

Microsserviços

Como vimos brevemente na aula anterior, a arquitetura de **Microsserviços** divide uma aplicação grande e complexa (monolito) em serviços menores e independentes.

- **Vantagens:** Cada microsserviço pode ser desenvolvido, testado, implantado (deploy) e escalado de forma independente. Eles se comunicam via rede (geralmente usando APIs REST).
- **Desafios:** O Java tradicional (criado para rodar por meses em servidores robustos) consumia muita memória e demorava a iniciar, o que não era ideal para esse novo mundo ágil e elástico da nuvem.

MicroProfile

Para resolver os desafios do Java no mundo dos microsserviços, surgiu o **Eclipse MicroProfile**. Ele é um conjunto de especificações focadas em otimizar o Enterprise Java para arquiteturas de microsserviços. Ele define padrões para coisas como: injeção de dependências, métricas, verificação de integridade (health checks) e tolerância a falhas. O Quarkus implementa muitas dessas especificações, trazendo o padrão de mercado para dentro do nosso projeto.

GraalVM

A **GraalVM** é uma máquina virtual de alta performance que suporta várias linguagens, mas seu grande trunfo para o Java é a capacidade de compilação **Ahead-of-Time (AOT)**.

- No Java clássico, o código é compilado para *bytecode* e, em tempo de execução, a JVM interpreta e compila para código de máquina (Just-in-Time ou JIT).
- Com a GraalVM (usando o *Native Image*), o nosso código Java é compilado diretamente para um executável binário nativo para o sistema operacional alvo **antes** da execução. O resultado? Aplicações que iniciam em milissegundos e consomem uma fração mínima de memória.

Parte 2: O Poder do Quarkus

O **Quarkus** (desenvolvido pela Red Hat) foi criado especificamente para a era dos contêineres e do Kubernetes. Seu lema é "Supersonic Subatomic Java". Ele junta o melhor do mundo Java estruturado com a leveza e velocidade exigidas pelos microsserviços.

Criando o Projeto

Para iniciar um projeto Quarkus, a forma mais comum é através do site `code.quarkus.io` ou via linha de comando utilizando o Maven. O Quarkus já configura toda a estrutura básica, os arquivos do Maven (`pom.xml`) e até mesmo um endpoint inicial para testarmos.

Adicionando Dependências (Extensões)

No mundo Quarkus, as bibliotecas são frequentemente chamadas de **Extensões**. Elas não são apenas `.jar` comuns; são otimizadas para trabalhar com a compilação nativa da GraalVM e para iniciar rapidamente.

- **On Demand:** A grande vantagem é que podemos adicionar extensões facilmente via linha de comando (ex:  `mvn quarkus:add-extension -Dextensions="nome-da-extensao"`) à medida que o nosso projeto precisa delas, sem precisar editar o `pom.xml` manualmente a todo momento.

Executando o Projeto Quarkus

A melhor experiência de desenvolvimento em Java hoje está no **Live Coding** do Quarkus. Quando rodamos o projeto com o comando  `mvn quarkus:dev` , o Quarkus entra em "Dev Mode".

- **O que isso significa?** Vocês podem alterar o código, salvar o arquivo e, ao recarregar a página no navegador, a alteração estará lá, instantaneamente. Não é necessário parar a aplicação, recompilar e subir o servidor novamente!

Parte 3: A Nossa Prática de Hoje

Na parte prática da nossa aula, vocês irão:

1. **Criar o projeto base** do zero utilizando as ferramentas do Quarkus.
2. **Adicionar as dependências iniciais** necessárias para construirmos a nossa API REST.
3. **Executar a aplicação em modo de desenvolvimento** e testar as mudanças em tempo real no código de vocês.

Preparem-se, pois a produtividade vai dar um salto a partir de agora!